NovaScript Compiler: A Domain-Specific Language for Data Pipelines

1. Abstract

This project presents the design and implementation of a Domain-Specific Language (DSL) named **NovaScript**, developed for simplifying data pipeline transformations. The compiler processes NovaScript programs through multiple stages: lexical analysis, syntax analysis, semantic analysis, intermediate representation (IR) generation, and target code generation.

NovaScript introduces high-level constructs such as load, filter, map, and output to define data workflows concisely. The compiler translates these programs into executable Python code using the panda's library.

The system also generates a **listing file** that displays source code with line numbers and error messages, ensuring robust error handling as required by the project guidelines. Additionally, a web-based interface enables visualization of each compilation stage, enhancing usability and demonstration capability.

2. Introduction

Data transformation pipelines are essential in modern applications, but implementing them using general-purpose programming languages often leads to verbose and complex code. Domain-Specific Languages (DSLs) address this by providing specialized syntax tailored to a specific domain.

NovaScript is designed as a DSL for **data pipeline processing**, allowing users to define transformations in a simple, readable format. It combines:

- SQL-like data manipulation
- Sequential execution similar to scripting languages

Example:

```
load users from "data.csv"
filter users where age > 30
map users to name = upper(name)
output users to "result.csv"
```

This project implements a complete compiler pipeline for NovaScript, demonstrating fundamental compiler concepts while producing practical, executable output.

3. Description of the Implemented Compiler

The NovaScript compiler is composed of several phases, each responsible for a specific task.

3.1 Lexical Analysis

The lexical analyzer scans the input program and converts it into tokens.

Token Types:

- Keywords: load, from, filter, where, map, to, output
- Identifiers: dataset and column names
- Literals: strings, numbers
- Operators: arithmetic and comparison operators
- Delimiters: parentheses, commas, semicolons

The lexer:

- Processes input character-by-character
- Tracks line and column numbers
- Handles comments (//)
- Generates ERROR tokens for invalid input

3.2 Syntax Analysis (Parser)

A recursive descent parser validates the syntax and constructs an Abstract Syntax Tree (AST).

Supported Statements:

- Load statement
- Filter statement
- Map statement
- Output statement

Expression Types:

- Identifiers
- Numbers
- Strings
- Function calls
- Binary expressions

Error recovery is implemented using synchronization, allowing parsing to continue after encountering errors.

3.3 Abstract Syntax Tree (AST)

The AST represents the structured form of the program.

Example:

```
Program
 ├── Load(users, "data.csv")
 ├── Filter(users, age > 30)
 ├── Map(users, name = upper(name))
 └── Output(users, "result.csv")
```

The AST enables easier semantic analysis and code generation.

3.4 Semantic Analysis

The semantic analyzer ensures logical correctness of the program.

Checks performed:

- Undeclared dataset usage
- Duplicate declarations
- Invalid function calls
- Type mismatches in expressions

Built-in functions:

- upper, lower, length, trim, abs, round

Errors are recorded with line numbers without terminating execution.

3.5 Intermediate Representation (IR)

The compiler generates **Three-Address Code (TAC)** as an intermediate representation.

Example:

```
t1 = LOAD "data.csv"
t2 = FILTER t1 WHERE age > 30
t3 = MAP t2 SET name = upper(name)
OUTPUT t3 TO "result.csv"
```

IR simplifies code generation and improves modularity.

3.6 Code Generation

The final stage converts the AST into executable Python code using pandas.

Example:

```
import pandas as pd

users = pd.read_csv("data.csv")
users = users[users["age"] > 30]
users["name"] = users["name"].str.upper()
users.to_csv("result.csv", index=False)
```

Code generation occurs only if no errors are present.

3.7 Compiler Pipeline Integration

The compilation process follows:

1. Lexical Analysis
2. Syntax Analysis
3. Semantic Analysis
4. IR Generation
5. Code Generation

All errors are collected and reported together.

3.8 Listing File Generation

The compiler produces a listing file showing:

- Source code with line numbers
- Errors associated with each line

Example:

```
2 | filter users where age >
  | ✗ [parser] Expected value after operator
```

This improves debugging and fulfills project requirements .

3.9 Web-Based Interface

A web-based interface was developed using React and Vite.

Features:

- Input NovaScript programs
- View tokens, AST, IR, and generated Python code
- Interactive debugging

This enhances usability and demonstration capability.

5. Result

The compiler was tested with multiple programs.

5.1 Valid Program

```
load users from "data.csv"
filter users where age > 30
```

map users to name = upper(name)
output users to "result.csv"

Output:

- Tokens generated correctly
- AST constructed
- IR generated
- Python code produced

5.2 Syntax Error

load users "data.csv"

Output:

- Parser error detected
- Listing file highlights missing keyword

5.3 Semantic Error

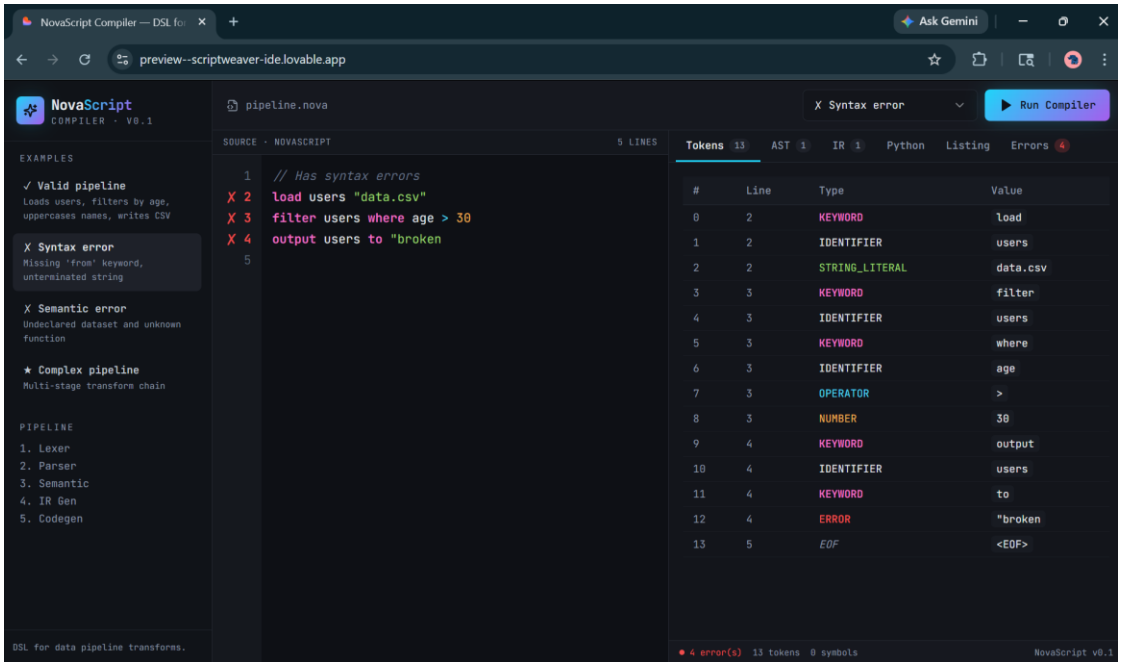
filter orders where total > 100

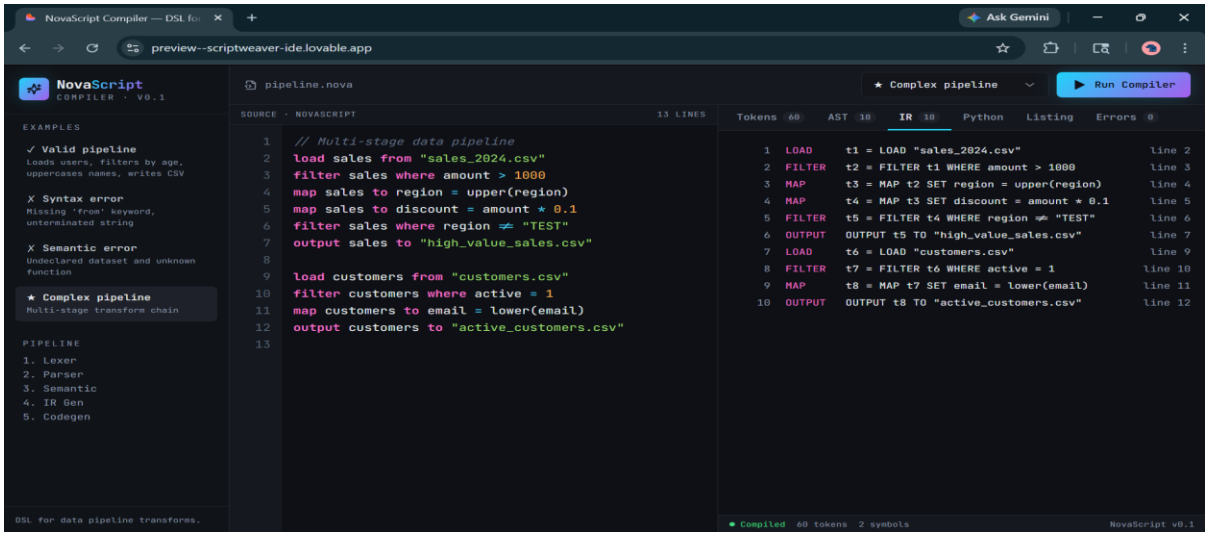
Output:

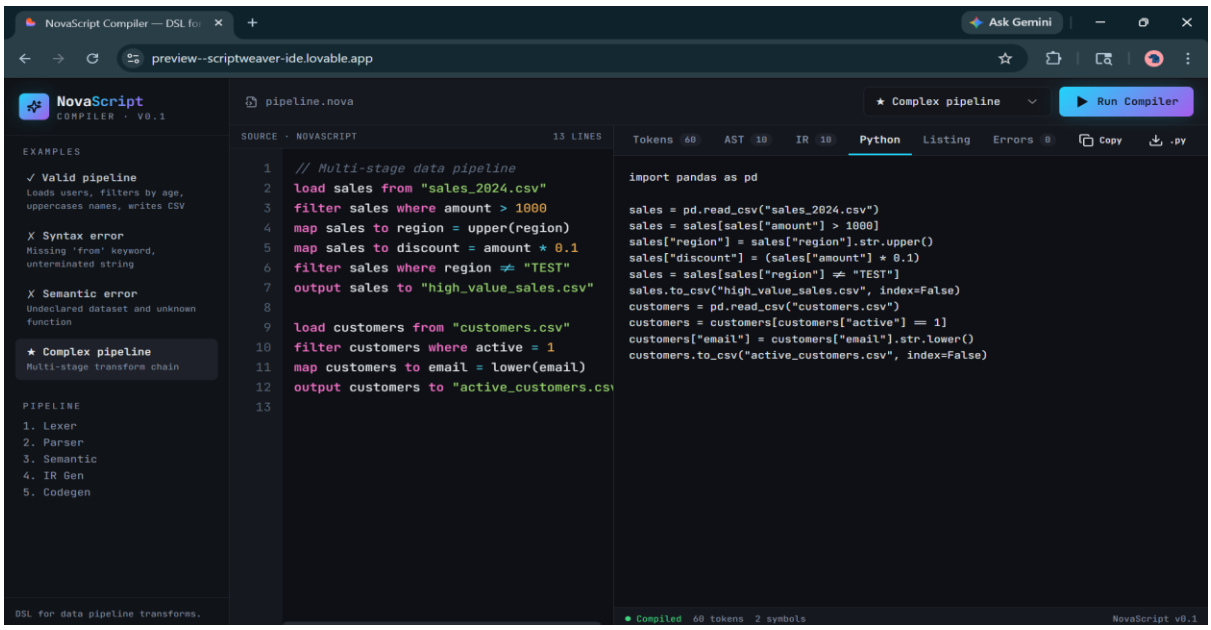
- Error: undeclared dataset

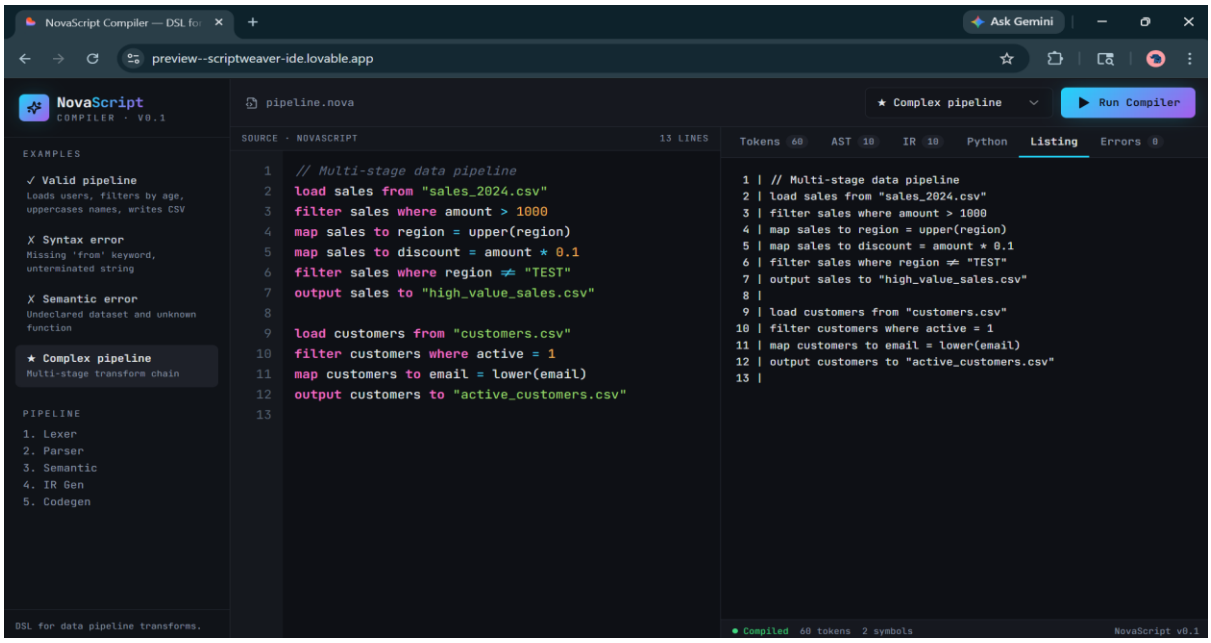
5.4 Complex Pipeline

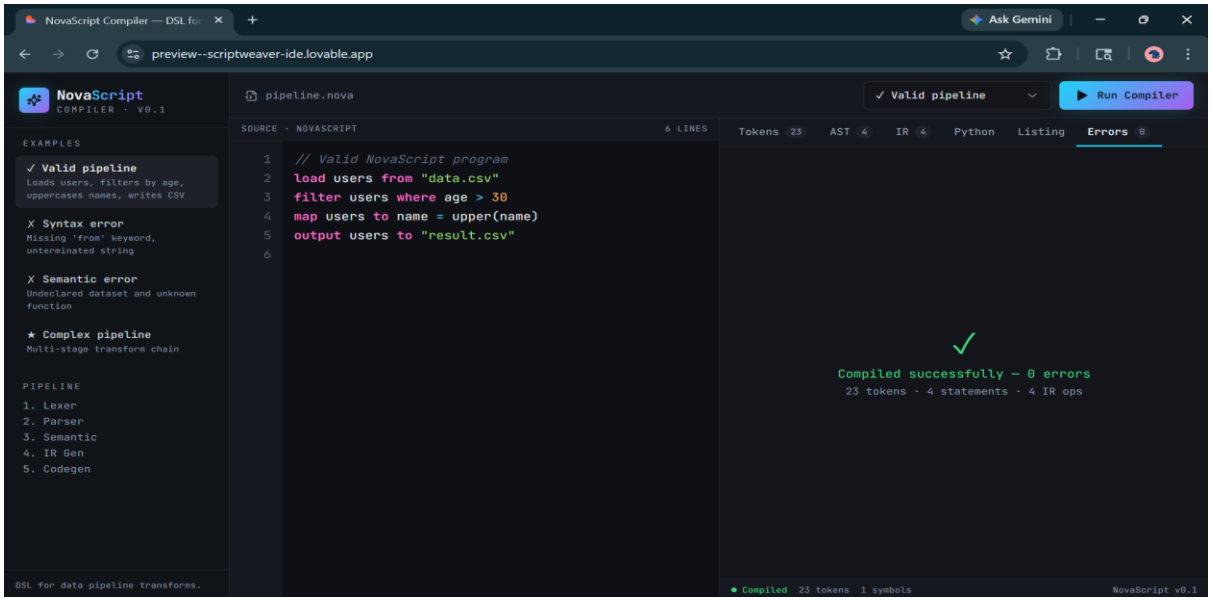
Multiple transformations across datasets were successfully processed, demonstrating scalability.

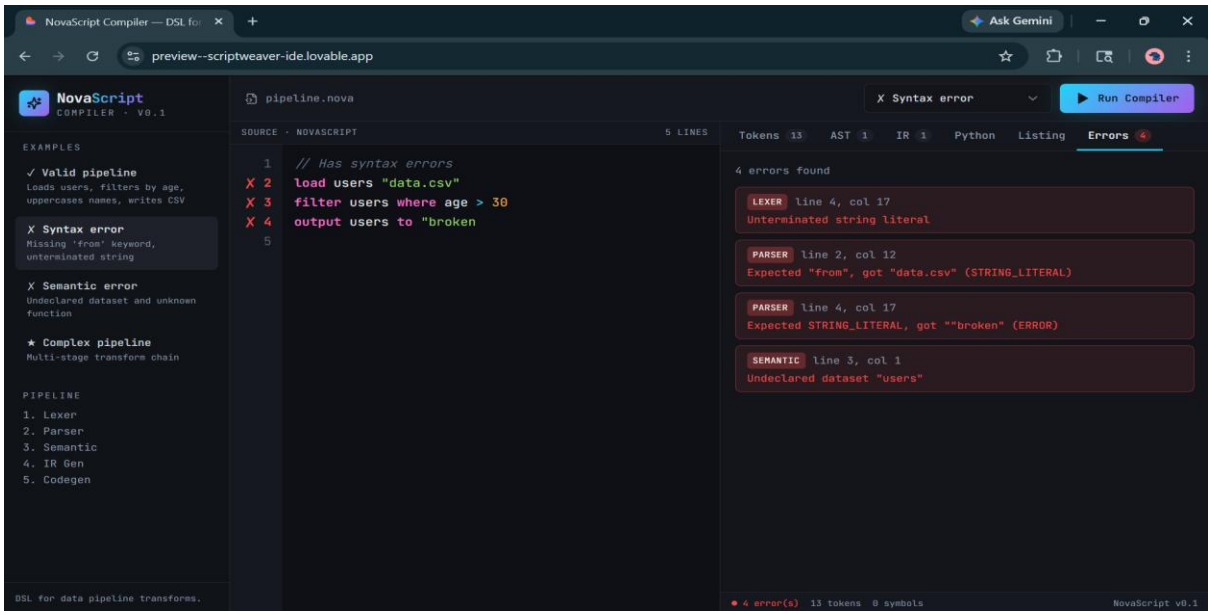


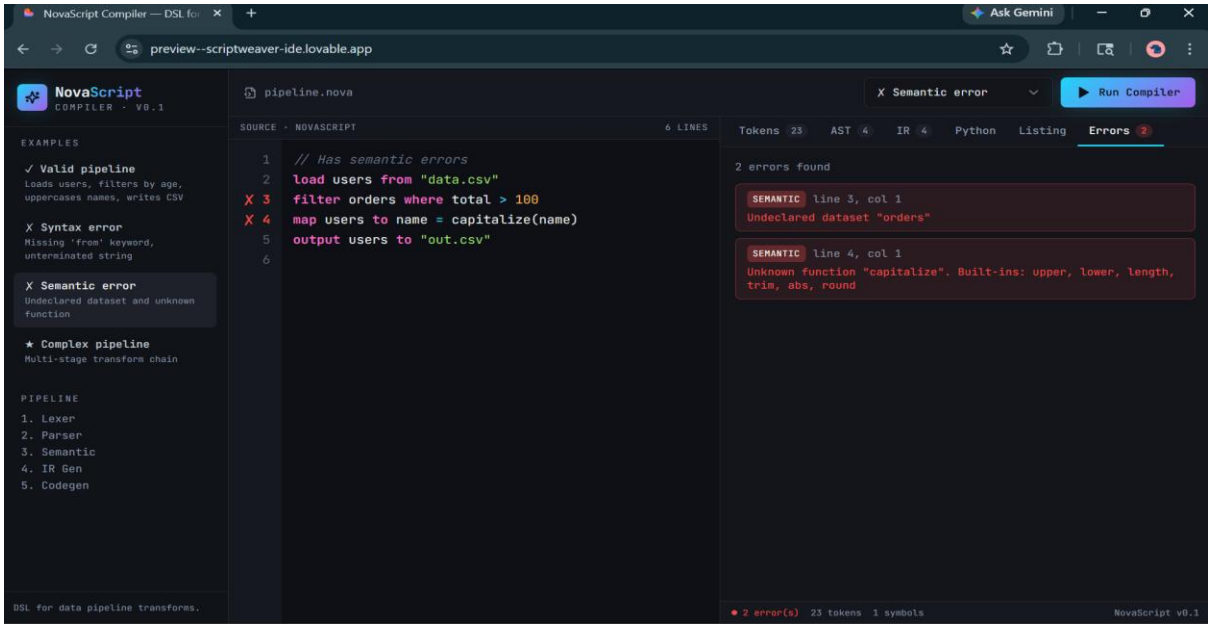

NovaScript Compiler — DSL for
Ask Gemini
preview--scriptweaver-ide.lovable.app
NovaScript
COMPILER · V0.1
pipeline.nova
★ Complex pipeline
Run Compiler
EXAMPLES
✓ Valid pipeline
Loads users, filters by age, uppercases names, writes CSV
✗ Syntax error
Missing 'from' keyword, unterminated string
✗ Semantic error
Undeclared dataset and unknown function
★ Complex pipeline
Multi-stage transform chain
PIPELINE
1. Lexer
2. Parser
3. Semantic
4. IR Gen
5. Codegen
SOURCE · NOVASCRIPT
13 LINES
// Multi-stage data pipeline
load sales from "sales_2024.csv"
filter sales where amount > 1000
map sales to region = upper(region)
map sales to discount = amount * 0.1
filter sales where region ≠ "TEST"
output sales to "high_value_sales.csv"
load customers from "customers.csv"
filter customers where active = 1
map customers to email = lower(email)
output customers to "active_customers.csv"
Tokens 60
AST 10
IR 10
Python
Listing
Errors 0
LOAD t1 = LOAD "sales_2024.csv" line 2
FILTER t2 = FILTER t1 WHERE amount > 1000 line 3
MAP t3 = MAP t2 SET region = upper(region) line 4
MAP t4 = MAP t3 SET discount = amount * 0.1 line 5
FILTER t5 = FILTER t4 WHERE region ≠ "TEST" line 6
OUTPUT OUTPUT t5 TO "high_value_sales.csv" line 7
LOAD t6 = LOAD "customers.csv" line 9
FILTER t7 = FILTER t6 WHERE active = 1 line 10
MAP t8 = MAP t7 SET email = lower(email) line 11
OUTPUT OUTPUT t8 TO "active_customers.csv" line 12
Compiled 60 tokens 2 symbols
NovaScript v0.1
DSL for data pipeline transforms.

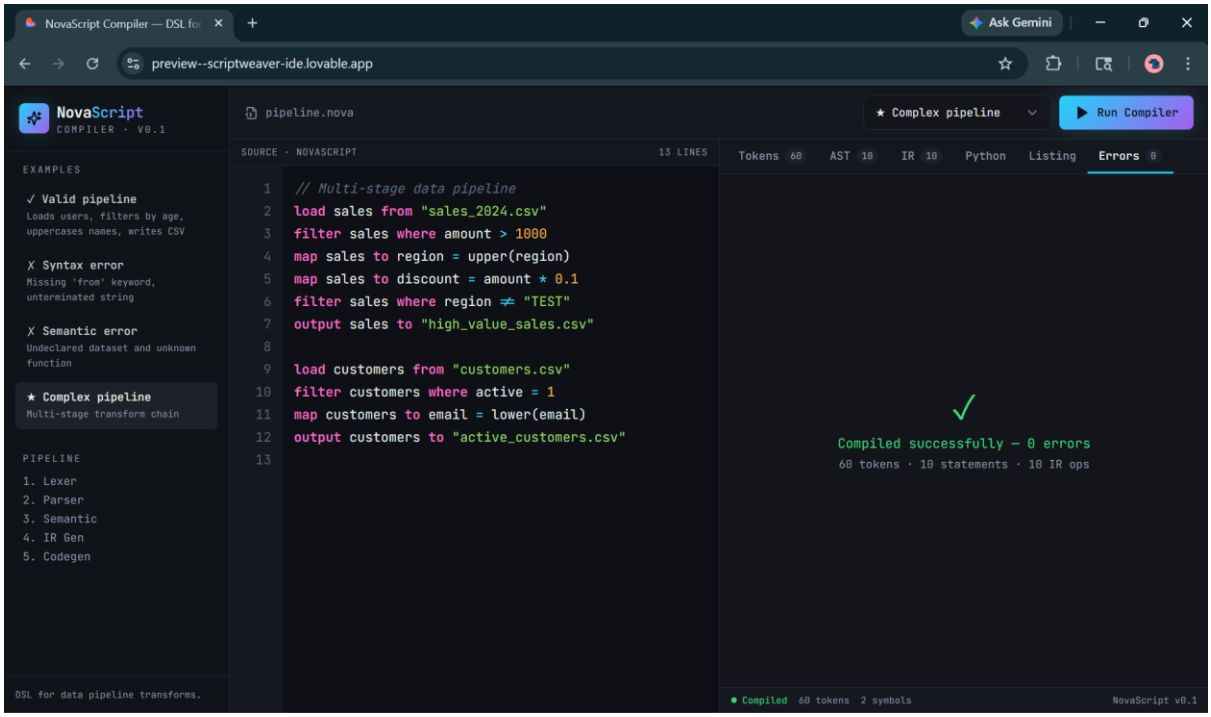

NovaScript Compiler — DSL for
Ask Gemini
preview--scriptweaver-ide.lovable.app
NovaScript
COMPILER · V0.1
pipeline.nova
★ Complex pipeline
Run Compiler
EXAMPLES
✓ Valid pipeline
Loads users, filters by age, uppercases names, writes CSV
✗ Syntax error
Missing 'from' keyword, unterminated string
✗ Semantic error
Undeclared dataset and unknown function
★ Complex pipeline
Multi-stage transform chain
PIPELINE
1. Lexer
2. Parser
3. Semantic
4. IR Gen
5. Codegen
SOURCE · NOVASCRIPT
13 LINES
// Multi-stage data pipeline
load sales from "sales_2024.csv"
filter sales where amount > 1000
map sales to region = upper(region)
map sales to discount = amount * 0.1
filter sales where region ≠ "TEST"
output sales to "high_value_sales.csv"
load customers from "customers.csv"
filter customers where active = 1
map customers to email = lower(email)
output customers to "active_customers.csv"
Tokens 60
AST 10
IR 10
Python
Listing
Errors 0
Copy
.py
import pandas as pd
sales = pd.read_csv("sales_2024.csv")
sales = sales[sales["amount"] > 1000]
sales["region"] = sales["region"].str.upper()
sales["discount"] = (sales["amount"] * 0.1)
sales = sales[sales["region"] ≠ "TEST"]
sales.to_csv("high_value_sales.csv", index=False)
customers = pd.read_csv("customers.csv")
customers = customers[customers["active"] == 1]
customers["email"] = customers["email"].str.lower()
customers.to_csv("active_customers.csv", index=False)
Compiled 60 tokens 2 symbols
NovaScript v0.1
DSL for data pipeline transforms.


NovaScript Compiler — DSL for
Ask Gemini
preview--scriptweaver-ide.lovable.app
NovaScript
COMPILER · V0.1
pipeline.nova
★ Complex pipeline
Run Compiler
EXAMPLES
✓ Valid pipeline
Loads users, filters by age, uppercases names, writes CSV
✗ Syntax error
Missing 'from' keyword, unterminated string
✗ Semantic error
Undeclared dataset and unknown function
★ Complex pipeline
Multi-stage transform chain
PIPELINE
1. Lexer
2. Parser
3. Semantic
4. IR Gen
5. Codegen
SOURCE · NOVASCRIPT
13 LINES
// Multi-stage data pipeline
load sales from "sales_2024.csv"
filter sales where amount > 1000
map sales to region = upper(region)
map sales to discount = amount * 0.1
filter sales where region ≠ "TEST"
output sales to "high_value_sales.csv"
load customers from "customers.csv"
filter customers where active = 1
map customers to email = lower(email)
output customers to "active_customers.csv"
Tokens 60
AST 10
IR 10
Python
Listing
Errors 0
1 | // Multi-stage data pipeline
2 | load sales from "sales_2024.csv"
3 | filter sales where amount > 1000
4 | map sales to region = upper(region)
5 | map sales to discount = amount * 0.1
6 | filter sales where region ≠ "TEST"
7 | output sales to "high_value_sales.csv"
8 |
9 | load customers from "customers.csv"
10 | filter customers where active = 1
11 | map customers to email = lower(email)
12 | output customers to "active_customers.csv"
13 |
Compiled 60 tokens 2 symbols
NovaScript v0.1
DSL for data pipeline transforms.


NovaScript Compiler — DSL for
Ask Gemini
preview--scriptweaver-ide.lovable.app
NovaScript
COMPILER · V0.1
EXAMPLES
✓ Valid pipeline
Loads users, filters by age, uppercases names, writes CSV
✗ Syntax error
Missing 'from' keyword, unterminated string
✗ Semantic error
Undeclared dataset and unknown function
★ Complex pipeline
Multi-stage transform chain
PIPELINE
1. Lexer
2. Parser
3. Semantic
4. IR Gen
5. Codegen
DSL for data pipeline transforms.
pipeline.nova
✓ Valid pipeline
Run Compiler
SOURCE · NOVASCRIPT
6 LINES
1 // Valid NovaScript program
2 load users from "data.csv"
3 filter users where age > 30
4 map users to name = upper(name)
5 output users to "result.csv"
6
Tokens 23
AST 4
IR 4
Python
Listing
Errors 0
✓
Compiled successfully — 0 errors
23 tokens · 4 statements · 4 IR ops
Compiled 23 tokens 1 symbols
NovaScript v0.1


NovaScript Compiler — DSL for
Ask Gemini
preview--scriptweaver-ide.lovable.app
NovaScript
COMPILER · V0.1
EXAMPLES
✓ Valid pipeline
Loads users, filters by age, uppercases names, writes CSV
✗ Syntax error
Missing 'from' keyword, unterminated string
✗ Semantic error
Undeclared dataset and unknown function
★ Complex pipeline
Multi-stage transform chain
PIPELINE
1. Lexer
2. Parser
3. Semantic
4. IR Gen
5. Codegen
DSL for data pipeline transforms.
pipeline.nova
✗ Syntax error
Run Compiler
SOURCE · NOVASCRIPT
5 LINES
1 // Has syntax errors
✗ 2 load users "data.csv"
✗ 3 filter users where age > 30
✗ 4 output users to "broken
5
Tokens 13
AST 1
IR 1
Python
Listing
Errors 4
4 errors found
LEXER line 4, col 17
Unterminated string literal
PARSER line 2, col 12
Expected "from", got "data.csv" (STRING_LITERAL)
PARSER line 4, col 17
Expected STRING_LITERAL, got ""broken" (ERROR)
SEMANTIC line 3, col 1
Undeclared dataset "users"
4 error(s) 13 tokens 0 symbols
NovaScript v0.1


NovaScript Compiler — DSL for
Ask Gemini
preview--scriptweaver-ide.lovable.app
NovaScript
COMPILER · V0.1
EXAMPLES
✓ Valid pipeline
Loads users, filters by age, uppercases names, writes CSV
✗ Syntax error
Missing 'from' keyword, unterminated string
✗ Semantic error
Undeclared dataset and unknown function
★ Complex pipeline
Multi-stage transform chain
PIPELINE
1. Lexer
2. Parser
3. Semantic
4. IR Gen
5. Codegen
DSL for data pipeline transforms.
pipeline.nova
✗ Semantic error
Run Compiler
SOURCE · NOVASCRIPT
6 LINES
1 // Has semantic errors
2 load users from "data.csv"
✗ 3 filter orders where total > 100
✗ 4 map users to name = capitalize(name)
5 output users to "out.csv"
6
Tokens 23
AST 4
IR 4
Python
Listing
Errors 2
2 errors found
SEMANTIC line 3, col 1
Undeclared dataset "orders"
SEMANTIC line 4, col 1
Unknown function "capitalize". Built-ins: upper, lower, length, trim, abs, round
2 error(s) 23 tokens 1 symbols
NovaScript v0.1



7. Conclusion

The NovaScript compiler demonstrates a complete implementation of a compiler pipeline for a domain-specific language. It successfully integrates lexical analysis, parsing, semantic validation, IR generation, and code generation.

The DSL simplifies data transformation tasks while maintaining clarity and usability. The addition of a web-based interface enhances understanding and usability of the compiler.

Future work can include optimization techniques, support for additional data operations, and integration with databases or distributed systems.